Benchmarking de CSV frente a Parquet con 15 millones de registros: cuello de botella de E/S y filtrado en la lectura

Angeles Daniela Caiza Rocha

Escuela Politécnica Nacional, Quito, Ecuador
Fundamentos de Big Data (IDSD413)
angeles.caiza@epn.edu.ec

Resumen Este informe compara un formato basado en filas (CSV) con uno columnar (Apache Parquet) usando un conjunto sintético de 15 millones de transacciones de retail, ejecutado en Google Colab con 12.7 GB de RAM. Se midieron tiempos de escritura y lectura, tamaño en disco, uso de memoria y dos consultas: una suma sobre una sola columna y un filtro doble (categoría *Juguetes* y estado *Completado*) que devuelve únicamente el monto. Parquet ocupó 3.29 veces menos espacio (225.39 MB frente a 740.53 MB), escribió 6.98 veces más rápido y resolvió el filtro doble 11.10 veces más rápido. Con 15 millones de filas no hubo agotamiento de memoria en ninguno de los dos formatos; la ventaja se concentró en tiempo y disco.

Keywords: Big Data · Apache Parquet · CSV · Predicate Pushdown · Column Pruning · Benchmarking

1. Introducción

En un análisis con muchos datos, el tiempo no se va tanto en el cálculo como en mover bytes entre el disco y la memoria. A ese problema se le llama cuello de botella de entrada/salida (E/S), y depende en buena parte de cómo está guardado el archivo. Un CSV almacena la tabla fila por fila, en texto plano: aunque solo se necesite una columna, hay que leer y convertir todo el contenido. Parquet guarda cada columna en su propio bloque, con tipos nativos y compresión, y por eso puede leer únicamente lo que la consulta pide [1].

Este laboratorio mide esa diferencia con un conjunto sintético de transacciones de retail, ejecutado en Google Colab con pandas y PyArrow. El Reto 1 sube el volumen a 15 millones de registros para ver qué pasa con la memoria al cargar cada formato. El Reto 2 resuelve una consulta con dos condiciones (categoría *Juguetes* y estado *Completado*) que devuelve solo el monto, para observar el efecto del filtrado durante la lectura (*predicate pushdown*) y de la lectura selectiva de columnas (*column pruning*).

El informe presenta primero la metodología y los resultados de cada reto, y cierra con una discusión de por qué se obtuvieron esas cifras y qué límites tiene la medición.

2. Metodología

Se usó el notebook base de la práctica en Google Colab (Python 3, pandas y PyArrow, 12.7 GB de RAM del sistema). El notebook genera un conjunto sintético de transacciones de retail con cinco columnas, entre ellas `monto`, `categoria` y `estado`, y lo guarda en CSV y en Parquet. Para el Reto 1 se subió el volumen a 15 millones de registros, y la corrida que se reporta se hizo desde una sesión recién reiniciada.

Se compararon ambos formatos en escritura, lectura completa y dos consultas: la suma de `monto` leyendo solo esa columna, y el filtro del Reto 2 (categoría Juguetes y estado Completado, devolviendo solo `monto`). En CSV se carga el archivo completo y se filtra en memoria; en Parquet se piden solo las columnas necesarias y se aplican los dos filtros en la lectura.

3. Reto 1: Test de Estrés de Memoria



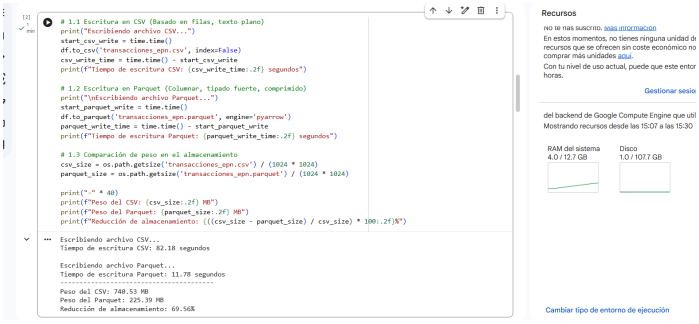

```
# Creación de un dataset sintético de transacciones de retail
np.random.seed(42)
df = pd.DataFrame({
    'id_transaccion': np.arange(N_ROWS),
    'id_cliente': np.random.randint(1, 100000, N_ROWS),
    'monto': np.random.uniform(5.0, 2500.0, N_ROWS),
    'categoria': np.random.choice(['Electronica', 'Ropa', 'Hogar', 'Alimentos', 'Juguetes'], N_ROWS),
    'estado': np.random.choice(['Completado', 'Pendiente', 'Cancelado', 'Reembolsado'], N_ROWS)
})

print(f"Dataset generado en {time.time() - start_time:.2f} segundos.")
df.info()

Generando 15000000 registros en memoria RAM...
Dataset generado en 19.29 segundos.
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000000 entries, 0 to 14999999
Data columns (total 5 columns):
 #   Column          Dtype
---  ------          -----
 0   id_transaccion  int64
 1   id_cliente      int64
 2   monto           float64
 3   categoria       object
 4   estado          object
dtypes: float64(1), int64(2), object(2)
memory usage: 572.2+ MB
```

Figura 1. Generación del conjunto de 15 000 000 de registros.

El conjunto se generó en 19.29 s. Según `df.info()` ocupa 572.2+ MB, pero el monitor de Colab pasó de 1.6 GB a 3.9 GB: las columnas de texto son de tipo `object` y `info()` no cuenta su peso real.




```
# 1.1 Escritura en CSV (Basado en filas, texto plano)
print("Escribiendo archivo CSV...")
start_csv_write = time.time()
df.to_csv('transacciones_epn.csv', index=False)
csv_write_time = time.time() - start_csv_write
print(f"Tiempo de escritura CSV: {csv_write_time:.2f} segundos")

# 1.2 Escritura en Parquet (Columnar, tipado fuerte, comprimido)
print("\nEscribiendo archivo Parquet...")
start_parquet_write = time.time()
df.to_parquet('transacciones_epn.parquet', engine='pyarrow')
parquet_write_time = time.time() - start_parquet_write
print(f"Tiempo de escritura Parquet: {parquet_write_time:.2f} segundos")

# 1.3 Comparación de peso en el almacenamiento
csv_size = os.path.getsize('transacciones_epn.csv') / (1024 * 1024)
parquet_size = os.path.getsize('transacciones_epn.parquet') / (1024 * 1024)

print("-" * 40)
print(f"Peso del CSV: {csv_size:.2f} MB")
print(f"Peso del Parquet: {parquet_size:.2f} MB")
print(f"Reducción de almacenamiento: {((csv_size - parquet_size) / csv_size) * 100:.2f}%")

Escribiendo archivo CSV...
Tiempo de escritura CSV: 82.18 segundos

Escribiendo archivo Parquet...
Tiempo de escritura Parquet: 11.78 segundos
----------------------------------------
Peso del CSV: 740.53 MB
Peso del Parquet: 225.39 MB
Reducción de almacenamiento: 69.56%
```

Figura 2. Tiempos de escritura y tamaño de los archivos.

Cuadro 1. Escritura y almacenamiento.

Métrica	CSV	Parquet	Relación
Tiempo de escritura (s)	82.18	11.78	6.98x
Tamaño en disco (MB)	740.53	225.39	3.29x
Reducción de espacio			69.56 %

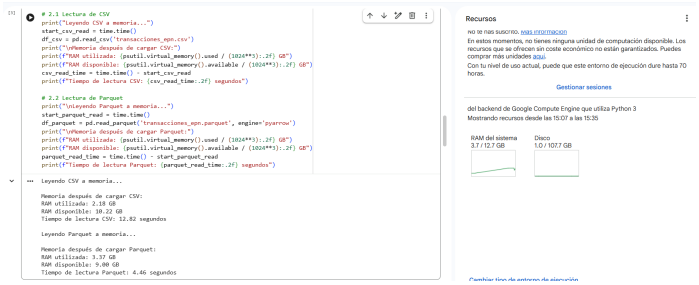


Figura 3. Lectura completa y memoria utilizada tras cada carga.

Cuadro 2. Lectura completa y memoria del sistema.

Métrica	CSV	Parquet
Tiempo de lectura (s)	12.82	4.46
RAM usada tras cargar (GB)	2.18	3.37
RAM disponible tras cargar (GB)	10.22	9.00

La sesión no se cayó con ninguno de los dos formatos: el uso máximo fue de 3.37 GB sobre 12.7 GB. La memoria se midió a nivel de sistema, por lo que las cifras son indicativas. En ambos casos la lectura completa termina en un DataFrame con las mismas cinco columnas, así que la ventaja de Parquet está en el tiempo (2.87x) y en el disco, no en la RAM final.

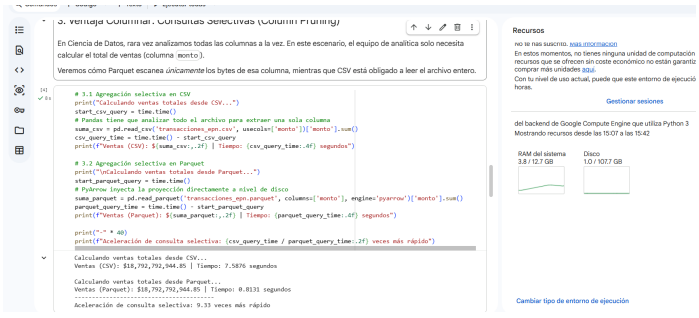


Figura 4. Suma de `monto` leyendo una sola columna.

Cuadro 3. Consulta selectiva: suma de `monto`.

Métrica	CSV	Parquet
Tiempo (s)	7.59	0.81
Total de ventas (USD)	18,792,792,944.85	18,792,792,944.85
Aceleración		9.33x

Ambos formatos devuelven la misma suma. Parquet lee solo el bloque de `monto`, mientras que el CSV recorre todo el archivo aunque se pida una columna.

4. Reto 2: Consulta Analítica Avanzada

La consulta pide el monto de las transacciones de categoría Juguetes con estado Completado, aplicando ambos filtros en la lectura.

```python
# 1. El método tradicional (CSV)
print("Filtrando CSV (Carga total -> Filtro)...")
start_csv = time.time()
df_temp_csv = pd.read_csv('transacciones_epn.csv')
df_filtrado_csv = df_temp_csv[
    (df_temp_csv['categoria'] == 'Juguetes')&
    (df_temp_csv['estado'] == 'Completado')
    ][['monto']]
tiempo_csv = time.time() - start_csv
print(f"Tiempo CSV: {tiempo_csv:.4f} segundos | Filas: {len(df_filtrado_csv)}")

# Liberamos memoria
del df_temp_csv, df_filtrado_csv
import gc; gc.collect()

# 2. El método Big Data (Parquet con Predicate Pushdown)
print("\nFiltrando Parquet (Filtro en disco -> Carga parcial)...")
start_parquet = time.time()
df_filtrado_parquet = pd.read_parquet(
    'transacciones_epn.parquet',
    engine='pyarrow',
    columns=['monto'],
    filters=[('categoria', '=', 'Juguetes'),
             ('estado', '=', 'Completado')  ]
)
tiempo_parquet = time.time() - start_parquet
print(f"Tiempo Parquet: {tiempo_parquet:.4f} segundos | Filas: {len(df_filtrado_parquet)}")

print("-" * 40)
print(f"Aceleración: {tiempo_csv / tiempo_parquet:.2f}x más rápido")
```

Figura 5. Código del filtro doble Juguetes y Completado (CSV y Parquet).

```
Filtrando CSV (Carga total -> Filtro)...
Tiempo CSV: 16.6263 segundos | Filas: 750894

Filtrando Parquet (Filtro en disco -> Carga parcial)...
Tiempo Parquet: 1.4974 segundos | Filas: 750894
----------------------------------------
Aceleración: 11.10x más rápido
```

Figura 6. Resultado del filtro doble: tiempos, filas y aceleración.

Cuadro 4. Filtro doble: Juguetes y Completado, devolviendo solo `monto`.

Métrica	CSV	Parquet
Tiempo (s)	16.63	1.50
Filas devueltas	750,894	750,894
Columnas devueltas	1	1
Columnas leídas del archivo	5	3
Aceleración		11.10x

Ambos métodos devuelven las mismas 750,894 filas (5.01 % del total). Parquet entrega una sola columna, pero lee tres para evaluar los filtros: `monto`, `categoria` y `estado`. El CSV carga las cinco y filtra después.

5. Discusión

Por qué gana Parquet. En escritura, el CSV convierte cada número a texto y lo guarda sin comprimir; Parquet guarda tipos nativos por columna, con codificación por diccionario para el texto repetido y compresión Snappy [1]. En lectura, el CSV obliga a analizar todo el texto, y Parquet salta directamente a los bloques de las columnas pedidas. La ventaja crece cuanto menos columnas se necesitan: 2.87x en lectura completa, 9.33x con una columna y 11.10x en el filtro doble, que lee tres.

Alcance del filtrado. La mejora del Reto 2 se explica sobre todo por leer 3 de 5 columnas y evitar el análisis del texto del CSV. Como los datos son aleatorios, es probable que cada grupo de filas de Parquet contenga todas las categorías y estados, y entonces las estadísticas no permitirían descartar bloques completos. No se revisaron los metadatos del archivo para comprobarlo.

Limitaciones. La memoria se midió a nivel de sistema, no del proceso, y cada prueba se ejecutó una sola vez.

6. Conclusiones

Con 15 millones de registros no se agotó la RAM en ninguno de los dos formatos (máximo 3.37 GB de 12.7 GB). La diferencia estuvo en la E/S: Parquet usó 3.29 veces menos disco, escribió 6.98 veces más rápido y consultó entre 2.87 y 11.10 veces más rápido según las columnas que necesitaba.

Para consultas que tocan pocas columnas de una tabla ancha, el formato columnar es la elección razonable. En una lectura completa la diferencia de tiempo se mantiene, pero la de RAM no.

7. Declaración del uso de Inteligencia Artificial

Se declara que se utilizaron herramientas de inteligencia artificial generativa como apoyo técnico y de redacción en la elaboración del informe y del documento en LaTeX: Claude (Anthropic, 2026). Los resultados experimentales provienen de la ejecución propia del notebook en Google Colab.

Referencias

1. Apache Software Foundation: Apache Parquet documentation. `https://parquet.apache.org/docs/` (consultado el 7 de noviembre de 2026)

2. Caiza Rocha, A.D.: Laboratorio Semana 6: Preparación de Datos y Benchmarking (CSV vs Parquet). Cuaderno de Google Colaboratory. https://colab.research.google.com/drive/1NJiQgToTxyXAuGbTDDtP2z9_G6yoU781?usp=sharing (consultado el 7 de noviembre de 2026)